SENTINEL: Security Event Network Triage Investigation with Neural Engine and Large Language Models

Author: Sivabalan T | Dept: Computer Science & Engineering, Sri Sai Ram Engineering College (TNEA Code: 1419)

Academic Year: 2026–2027 | **Repository:** github.com/siva2526k-art/SENTINEL | **Target:** Hac'KP 2026 Specification

1. ABSTRACT / EXECUTIVE SUMMARY

Security Operations Centers (SOCs) face acute operational bottlenecks driven by overwhelming telemetry volume, high false-positive rates, and analyst burnout. In typical enterprise environments, security analysts spend between 30 and 45 minutes investigating individual alerts, leading to significant delays and leaving a major portion of security notifications unreviewed. Concurrently, while Generative Artificial Intelligence and Large Language Models (LLMs) offer advanced natural-language reasoning for threat analysis, transmitting raw security telemetry to commercial cloud APIs creates severe data privacy vulnerabilities and risks exposing sensitive credentials, internal IP topographies, and employee data to external endpoints.

This paper presents **SENTINEL** (*Security Event Network Triage Investigation with Neural Engine and Large Language Models*), an open-source research prototype designed to evaluate a privacy-aware, human-supervised approach to automated SOC alert triage. SENTINEL implements an asynchronous SIEM syslog ingestion bridge, a local zero-trust regex sanitizer with an inline prompt-injection firewall, a three-tier AI routing module, an embedded vector threat memory store (ChromaDB), a temporal entity correlator with directed attack-graph construction, an Abstract Syntax Tree (AST) Python code execution sandbox, human-in-the-loop (HITL) authorization gates, mock-mode active response controllers, JSONL audit logging, and automated executive PDF incident report generation via ReportLab.

By retaining token de-anonymization lookup dictionaries strictly within volatile local RAM, SENTINEL prevents sensitive network identifiers from traversing external boundaries during AI triage. SENTINEL is currently implemented as an early-stage Minimum Viable Product (MVP) to demonstrate architectural feasibility. Future work will focus on empirical evaluation against labeled SIEM datasets, production-grade security hardening, and formal bench-testing prior to operational deployment.

Keywords: Privacy-Preserving AI, SOC Alert Triage, Multi-Tier Model Cascading, Zero-Trust Sanitization, MITRE ATT&CK;, Attack Graphs, AST Code Sandbox, Human-in-the-Loop, Digital Forensics.

2. PROBLEM STATEMENT

Modern enterprise, municipal, and government computer networks rely on SIEM platforms—such as Wazuh, Elastic, or Splunk—to aggregate logs. However, security teams face three fundamental systemic challenges:

1. **Alert Fatigue & Manual Triage Delay:** Rule-based detection signatures trigger thousands of notifications daily. Tier-1 analysts spend 30 to 45 minutes per alert, causing severe cognitive burnout and extended attacker dwell time.
2. **Data Privacy & Telemetry Leakage Risks:** Ingesting un-scrubbed security logs into commercial cloud LLM APIs introduces compliance risks. Telemetry contains Personal Identifiable Information (PII), employee emails, internal IPs (RFC 1918), MAC addresses, and API/JWT tokens that violate statutory data privacy mandates.
3. **Requirement for Offline & Air-Gapped Operation:** Defense networks, law enforcement labs, and critical infrastructure operate within strict air-gapped physical boundaries requiring local, self-hosted AI reasoning capabilities.

3. PROPOSED SYSTEM

SENTINEL is engineered as a modular Python framework incorporating the following core technical components:
• **Wazuh / SIEM Webhook Ingestion** (src/ingestion/wazuh_listener.py): Non-blocking asynchronous FastAPI listener.
• **Local Zero-Trust Data Sanitizer** (src/sanitizer.py): Regex pattern matching scrubbing IPs, emails, MACs, and API tokens (e.g., [USER_1], [INTERNAL_IP_1]), holding lookup tables in volatile RAM.
• **Prompt Injection Firewall Guard** (src/sanitizer.py): Detects adversarial prompt overrides, replacing them with a [NEUTRALIZED_PROMPT_INJECTION] marker.
• **Three-Tier AI Router** (src/router.py, src/ai_client.py): Routes queries to local Tier 1 Ollama (defaulting to deepseek-r1:8b with a llama3.2:1b fallback) or policy-controlled cloud endpoints (Groq, Gemini, OpenRouter, OpenAI).
• **MITRE ATT&CK; Mapper** (src/mitre_mapper.py): Correlates log attributes with tactics (e.g., Credential Access) and technique IDs (e.g., T1110 Brute Force).
• **Vector Threat Memory / RAG** (src/memory.py): Embedded ChromaDB store calculating cosine similarity against historical incidents.
• **Entity, Temporal & Attack-Graph Correlation** (src/correlation/): Clusters events into Directed Acyclic Graphs (DAGs).
• **AST Code Execution Sandbox** (src/sandbox.py): Inspects script syntax trees with ast.parse(), blocking dangerous primitives (os, sys, subprocess, eval).
• **FastAPI REST API** (src/api/main.py): Application gateway for REST and real-time WebSockets integration.
• **Human-in-the-Loop Authorization Gate** (src/response/response_engine.py): Requires explicit analyst sign-off before containment actions proceed.
• **Mock-Mode Active Response Controllers** (src/response/): Simulated containment modules (firewall, process kill, host isolation).
• **JSONL Audit Logger** (src/audit_logger.py): File-backed audit logger (sentinel_audit_trail.jsonl).
• **Executive PDF Generator** (src/reports/pdf_generator.py): Compiles ReportLab PDF incident summaries.
• **Discord SOC Notifier** (src/integrations/discord_bot.py): Dispatches real-time alerts and HITL approval pings.

4. END-TO-END SYSTEM ARCHITECTURE

Stage 1: Ingestion & Privacy Boundary: Telemetry arrives via FastAPI syslog webhooks. DataSanitizer neutralizes prompt injections and tokenizes PII via regex. Lookup tables reside exclusively in volatile RAM.
Stage 2: Context, RAG, Correlation & Attack Graph: Sanitized logs are mapped to MITRE ATT&CK; TTPs, queried against ChromaDB embeddings, and clustered into Directed Acyclic Graphs (DAGs).
Stage 3: AI Triage, Human Approval, Controlled Response, Audit & Reporting: SentinelRouter dispatches to Tier-1 local Ollama or cloud endpoints. AST Code Sandbox inspects de-obfuscation scripts. Containment recommendations require analyst HITL authorization before mock response execution. Audit milestones append to JSONL, and ReportLab compiles incident PDFs.

5. CURRENT PROTOTYPE STATUS

SENTINEL is an early-stage **research prototype and Minimum Viable Product (MVP)** designed to evaluate privacy-preserving AI triage workflows. It is **not** a production-ready SOC platform, commercial SOAR software, or verified legal forensics tool. Current capabilities represent architectural proofs-of-concept.

6. LIMITATIONS AND FUTURE WORK

Current Limitations:
1. *Sanitizer Scope & RAM Security:* Relies on Regex rather than Named Entity Recognition (NER); RAM identity maps are stored in unencrypted volatile memory.
2. *Security Hardening & RBAC:* Lacks production-grade Role-Based Access Control (RBAC), multi-factor authentication, and encrypted secret stores.
3. *Audit Log Verification:* Audit records are stored in a standard local JSONL text file without cryptographic signatures or hardware tamper-evident enforcement.
4. *Model Output Schema & Containment Controls:* Model outputs require strict JSON schema validation. Active containment runs in mock mode by default.

Future Work Roadmap:
• Construct a labeled benchmark dataset using Wazuh SIEM logs to measure triage accuracy, latency, and false-positive reduction.
• Integrate lightweight local NER models (spaCy / ONNX) and encrypt RAM lookup tables.
• Implement JWT-based RBAC, TLS-encrypted webhooks, tamper-evident audit logs, mandatory IP allowlists, and containerization (Docker/CI/CD).

7. CONCLUSION

SENTINEL demonstrates a privacy-preserving, human-supervised approach to AI-assisted SIEM alert triage. By combining local regex data sanitization, prompt-injection neutralization, three-tier model routing, vector threat memory, and mandatory human authorization gates, the framework illustrates how organizations can leverage language models while retaining control over sensitive telemetry. SENTINEL requires rigorous, reproducible benchmark evaluation and extensive security hardening before any real-world operational deployment.

8. REFERENCES

1. **MITRE ATT&CK; Framework:** MITRE Corporation, 'MITRE ATT&CK; Enterprise Matrix,' 2024. Available: attack.mitre.org
2. **Wazuh Open Source SIEM:** Wazuh Inc., 'Wazuh Documentation & Active Response Architecture,' 2024. Available: documentation.wazuh.com
3. **ChromaDB Vector Store:** Chroma Core Inc., 'Chroma: The Open-Source Embedding Database,' 2024. Available: docs.trychroma.com
4. **FastAPI Framework:** S. Ramírez, 'FastAPI High Performance Web Framework,' 2024. Available: fastapi.tiangolo.com
5. **Ollama Local LLM Runtime:** Ollama Project, 'Ollama: Get up and running with Llama 3.2 and DeepSeek locally,' 2024. Available: ollama.com
6. **ReportLab PDF Library:** ReportLab Software Ltd., 'ReportLab Open Source PDF Toolkit,' 2024. Available: www.reportlab.com